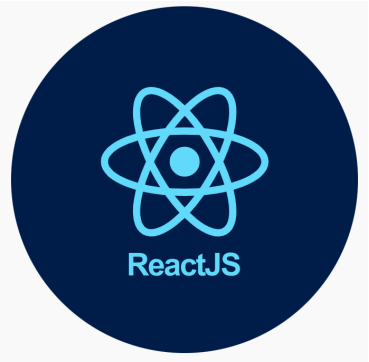


Atelier 4 - ReactJS

Chargement des données et traitement des objets

Objectifs de l'atelier

À la fin de cet atelier, vous saurez :

1. créer des composants réutilisables et leur transmettre des données avec les **props** ;
2. charger une liste d'objets depuis un fichier et l'afficher avec `map()` ;
3. gérer un état local avec le hook `useState` (afficher / cacher) ;
4. réagir aux événements (`onClick`) et ouvrir des liens externes ;
5. créer un formulaire contrôlé et afficher une alerte avec SweetAlert2.

Prérequis : Node.js installé, bases de HTML/CSS et de JavaScript (objets, tableaux, fonctions fléchées).

Résultat attendu : une page qui affiche une grille de cartes (une carte par merveille naturelle). Chaque carte contient une image, une description, un bouton pour afficher les détails, un lien Wikipédia et un lien Google Maps. Un formulaire de connexion est ajouté en bas de la page.

Étape 1 : Mise en place du projet

1. Créer un nouveau projet React avec Vite :

```
npm create vite@latest natural-wonders -- --template react
cd natural-wonders
npm install
```

2. Installer Bootstrap :

```
npm install bootstrap
```

3. Importer Bootstrap dans `src/main.jsx` (en haut du fichier) :

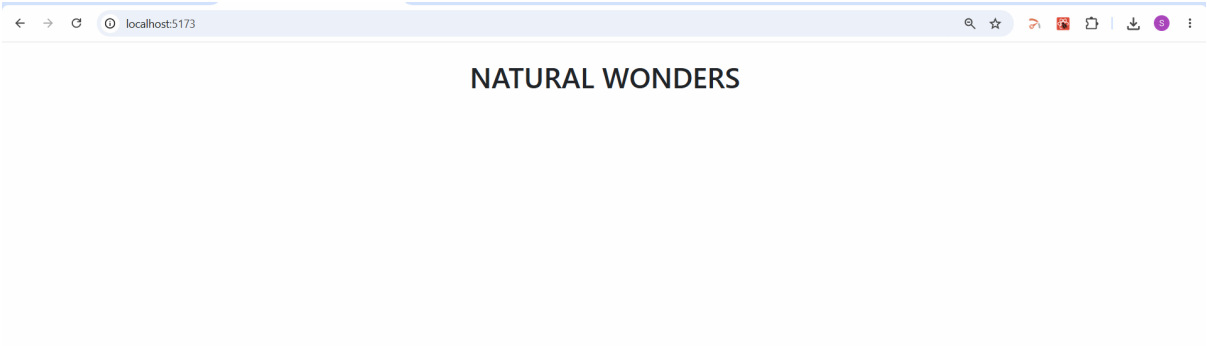
```
import 'bootstrap/dist/css/bootstrap.min.css';
```

4. Remplacer le contenu de `src/App.jsx` par un simple titre, puis lancer le projet avec `npm run dev` :

src/App.jsx

```
function App() {
  return (
    <div className="container my-4">
      <h1 className="text-center mb-4">NATURAL WONDERS</h1>
    </div>
  );
}

export default App;
```



Images

Créer un dossier `public/images` et y placer une image par merveille (grand-canyon.jpg, victoria.jpg, barriere-corail.jpg, everest.jpg, halong.jpg, iguazu.jpg). Tout fichier du dossier `public` est accessible depuis la racine du site : `/images/everest.jpg`.

On peut aussi utiliser directement l'URL d'une image en ligne.

Étape 2 : Premier composant Card et props

Un **composant** est une fonction JavaScript qui retourne du JSX (du HTML dans du JavaScript). Les **props** sont les données que le composant parent transmet au composant enfant : elles rendent le composant réutilisable.

2.1 Créer le composant

Sous `src`, créer le dossier `components`, puis le fichier `Card.jsx`. Le code est inspiré de la card Bootstrap (https://getbootstrap.com/docs/5.3/components/card/).

src/components/Card.jsx

```
function Card(props) {
  return (
    <div className="card" style={{ width: "18rem" }}>
      <img src={props.image} className="card-img-top" alt={props.name} />
      <div className="card-body">
        <h5 className="card-title">{props.name}</h5>
        <p className="card-text">{props.desc}</p>
      </div>
    </div>
  );
}

export default Card;
```

Règles JSX à retenir (différences avec le HTML)

- `class` devient `className` (sinon un warning apparaît dans la console).
- `style="width: 18rem;"` devient `style={{ width: "18rem" }}` : un objet JavaScript entre accolades.
- Les balises sans contenu doivent être fermées : `<img ... />`, `<br />`.
- Les accolades `{ }` permettent d'insérer une expression JavaScript : `{props.name}`.
- `props` est un objet JS : ici il possède 3 attributs (name, desc, image).

2.2 Utiliser le composant dans App

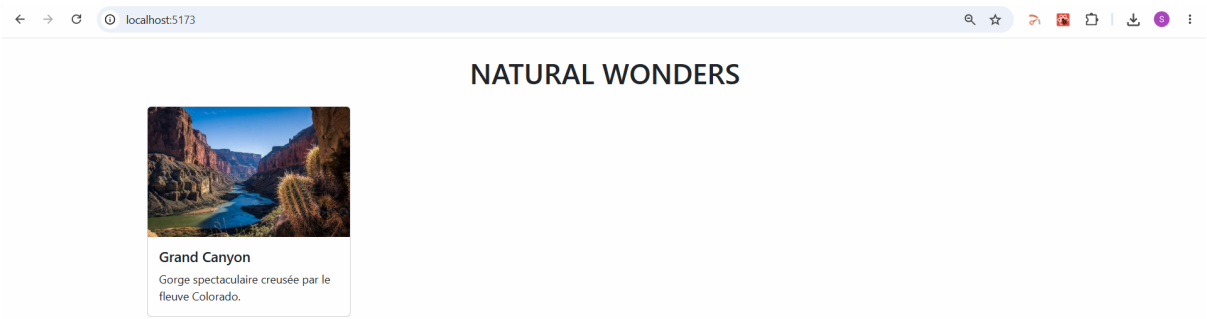
Dans `App.jsx`, importer `Card` et lui transmettre les données sous forme d'attributs :

src/App.jsx

```
import Card from "./components/Card";

function App() {
  return (
    <div className="container my-4">
      <h1 className="text-center mb-4">NATURAL WONDERS</h1>
      <Card
        name="Grand Canyon"
        desc="Gorge spectaculaire creusée par le fleuve Colorado."
        image="/images/grand-canyon.jpg"
      />
    </div>
  );
}

export default App;
```



On peut réutiliser `<Card />` autant de fois qu'on veut avec des valeurs différentes. Mais écrire chaque carte à la main n'est pas pratique : on va plutôt placer les données dans un fichier.

Étape 3 : Charger les données depuis un fichier

En général, les données sont séparées de l'affichage. On crée un tableau d'objets dans un fichier `Data.jsx`. Chaque objet possède un `id` unique.

Créer le fichier `src/Data.jsx` (au même niveau que `App.jsx`) :

src/Data.jsx

```
export const WondersArr = [
  {
    id: 1,
    name: "Grand Canyon",
    desc: "Gorge spectaculaire creusée par le fleuve Colorado pendant des millions
d'années. Ses parois multicolores racontent près de deux milliards d'années d'histoire
géologique.",
    image: "/images/grand-canyon.jpg",
```

```
    location: "Arizona, États-Unis",
    type: "Canyon",
    size: "Environ 446 km de long",
    google_map: "https://www.google.com/maps/search/?api=1&query=Grand+Canyon",
  },
  {
    id: 2,
    name: "Chutes Victoria",
    desc: "Situées sur le fleuve Zambèze, elles forment l'un des plus grands rideaux
d'eau du monde. Les habitants les appellent « Mosi-oa-Tunya » : la fumée qui gronde.",
    image: "/images/victoria.jpg",
    location: "Zambie / Zimbabwe",
    type: "Chutes d'eau",
    size: "1,7 km de large, 108 m de haut",
    google_map: "https://www.google.com/maps/search/?api=1&query=Victoria+Falls",
  },
  {
    id: 3,
    name: "Grande Barrière de corail",
    desc: "Plus grand récif corallien de la planète, visible depuis l'espace. Il abrite
des milliers d'espèces de poissons, de coraux et de mollusques.",
    image: "/images/barriere-corail.jpg",
    location: "Queensland, Australie",
    type: "Récif corallien",
    size: "Environ 2 300 km de long",
    google_map: "https://www.google.com/maps/search/?api=1&query=Great+Barrier+Reef",
  },
  {
    id: 4,
    name: "Mont Everest",
    desc: "Point culminant de la Terre, situé dans la chaîne de l'Himalaya. Il attire
chaque année des alpinistes venus du monde entier.",
    image: "/images/everest.jpg",
    location: "Népal / Chine",
    type: "Montagne",
    size: "8 849 m d'altitude",
    google_map: "https://www.google.com/maps/search/?api=1&query=Mount+Everest",
  },
  {
    id: 5,
    name: "Baie d'Along",
    desc: "Baie parsemée de près de 2 000 îlots calcaires couverts de végétation, qui
émergent des eaux émeraude du golfe du Tonkin.",
    image: "/images/halong.jpg",
    location: "Vietnam",
    type: "Baie / relief karstique",
    size: "Environ 1 553 km²",
    google_map: "https://www.google.com/maps/search/?api=1&query=Ha+Long+Bay",
  },
  {
    id: 6,
    name: "Chutes d'Iguazú",
    desc: "Ensemble d'environ 275 cascades situé à la frontière entre l'Argentine et le
Brésil, au cœur d'une forêt tropicale protégée.",
    image: "/images/iguazu.jpg",
    location: "Argentine / Brésil",
    type: "Chutes d'eau",
    size: "Environ 2,7 km de large",
    google_map: "https://www.google.com/maps/search/?api=1&query=Iguazu+Falls",
  },
];
```

Dans `App.jsx`, importer le tableau puis le parcourir avec `map()` pour générer une carte par élément. On transmet cette fois **l'objet entier** via une seule prop `wonder`. Les classes Bootstrap `row row-cols-md-3 g-4` affichent les cartes en grille de 3 colonnes.

src/App.jsx

```
import Card from "./components/Card";
import { WondersArr } from "./Data";

function App() {
  return (
    <div className="container my-4">
      <h1 className="text-center mb-4">NATURAL WONDERS</h1>
      <div className="row row-cols-1 row-cols-md-3 g-4">
        {WondersArr.map((wonder) => (
          <Card key={wonder.id} wonder={wonder} />
        ))}
      </div>
    </div>
  );
}

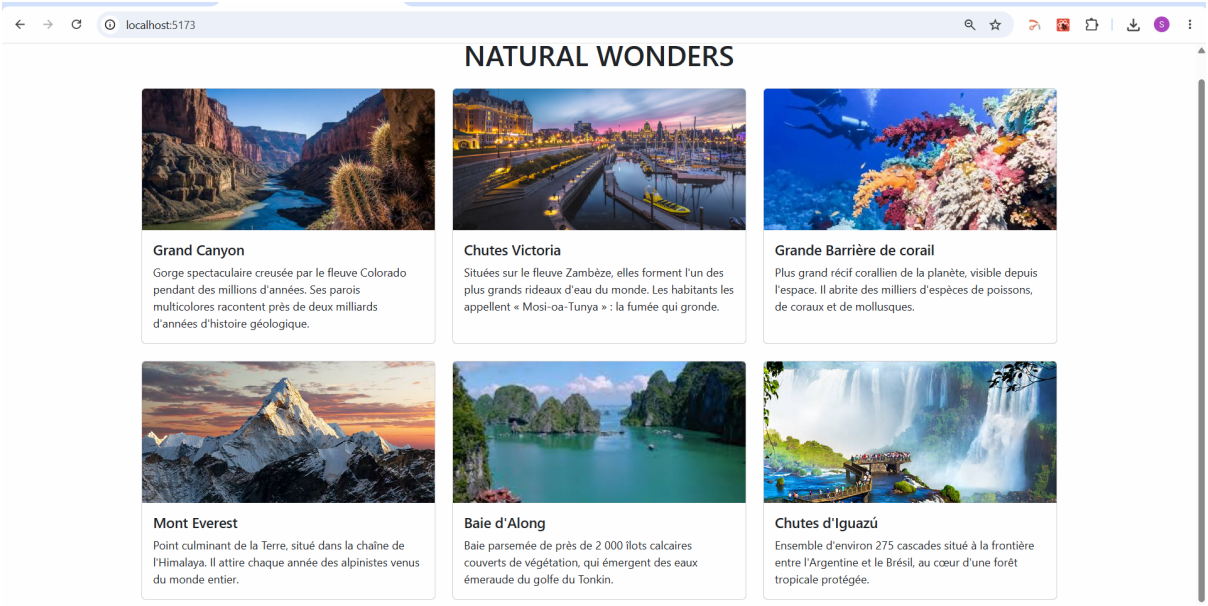
export default App;
```

Mettre à jour `Card.jsx` pour lire l'objet `wonder`. On utilise la **déstructuration** `({ wonder })` pour récupérer directement la prop :

src/components/Card.jsx

```
function Card({ wonder }) {
  return (
    <div className="col">
      <div className="card h-100">
        <img src={wonder.image} className="card-img-top" alt={wonder.name}
             height={200} style={{ objectFit: "cover" }} />
        <div className="card-body">
          <h5 className="card-title">{wonder.name}</h5>
          <p className="card-text">{wonder.desc}</p>
        </div>
      </div>
    </div>
  );
}

export default Card;
```



À retenir

- `key` est obligatoire dans une liste générée par `map()` : elle permet à React d'identifier chaque élément. On utilise l'`id`.
- `function Card({ wonder })` est équivalent à `function Card(props)` puis `props.wonder`. Les deux écritures sont correctes.

Étape 4 : Composant Details (props de type objet)

On souhaite afficher 3 informations supplémentaires (localisation, type, dimension) dans un composant séparé `Details`, imbriqué dans `Card`.

Créer le fichier `src/components/Details.jsx` :

src/components/Details.jsx

```
function Details({ informations }) {
  const { location, type, size } = informations;

  return (
    <ul className="list-group">
      <li className="list-group-item"><strong>Localisation :</strong> {location}</li>
      <li className="list-group-item"><strong>Type :</strong> {type}</li>
      <li className="list-group-item"><strong>Dimension :</strong> {size}</li>
    </ul>
  );
}

export default Details;
```

Dans `Card.jsx`, construire l'objet `informations` puis le transmettre à `Details` :

src/components/Card.jsx (extrait mis à jour)

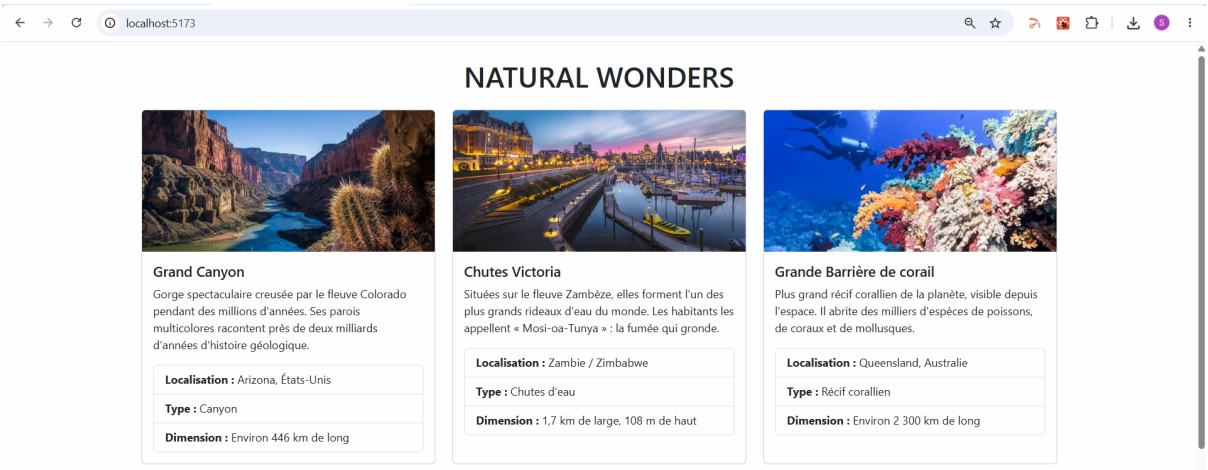
```
import Details from "./Details";

function Card({ wonder }) {
```

```
    const informations = {
      location: wonder.location,
      type: wonder.type,
      size: wonder.size,
    };

    return (
      <div className="col">
        <div className="card h-100">
          <img src={wonder.image} className="card-img-top" alt={wonder.name}
               height={200} style={{ objectFit: "cover" }} />
          <div className="card-body">
            <h5 className="card-title">{wonder.name}</h5>
            <p className="card-text">{wonder.desc}</p>
            <Details informations={informations} />
          </div>
        </div>
      </div>
    );
  }

  export default Card;
```



Étape 5 : Afficher / cacher les détails avec useState

Un **état** (state) est une donnée propre au composant qui peut changer. Quand l'état change, React ré-affiche automatiquement le composant. Avec les composants fonctionnels, on utilise le hook `useState` :

```
  const [showDetails, setShowDetails] = useState(false);
```

- `showDetails` : la valeur actuelle de l'état (ici `false` au départ) ;
- `setShowDetails` : la fonction qui permet de modifier cette valeur.

On ajoute un bouton dont le texte est « Show details » ou « Hide details » selon l'état :

src/components/Details.jsx (version finale)

```
  import { useState } from "react";

  function Details({ informations }) {
    const { location, type, size } = informations;
```

```
    const [showDetails, setShowDetails] = useState(false);

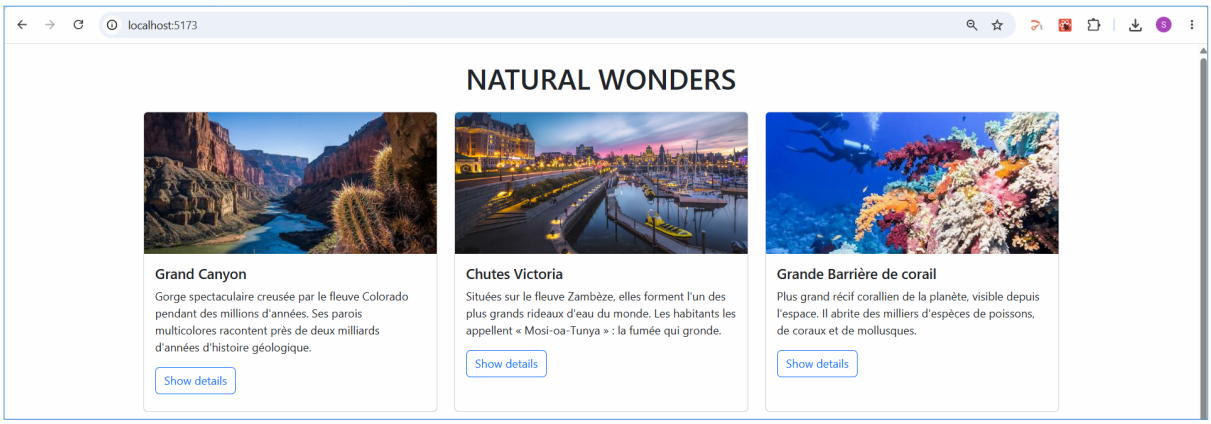
    return (
      <>
        <button className="btn btn-outline-primary mb-2"
                onClick={() => setShowDetails(!showDetails)}>
          {showDetails ? "Hide details" : "Show details"}
        </button>

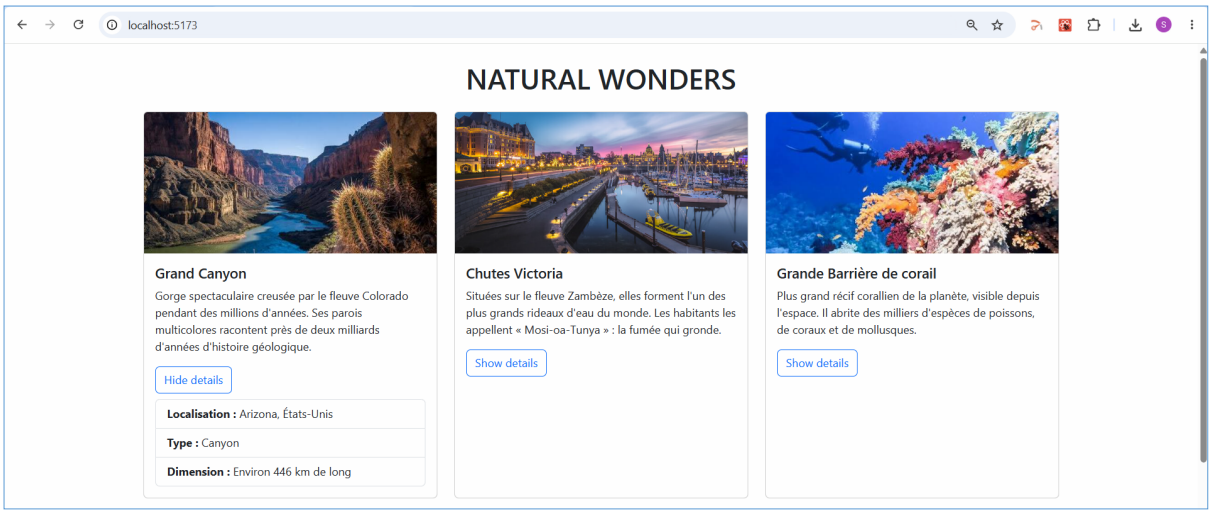
        {showDetails && (
          <ul className="list-group">
            <li className="list-group-item"><strong>Localisation :</strong> {location}</li>
            <li className="list-group-item"><strong>Type :</strong> {type}</li>
            <li className="list-group-item"><strong>Dimension :</strong> {size}</li>
          </ul>
        )}
      </>
    );
  }

  export default Details;
```

Explications

- `!showDetails` inverse la valeur : `true` devient `false` et inversement.
- `condition ? A : B` (opérateur ternaire) choisit le texte du bouton.
- `condition && <JSX />` (rendu conditionnel) n'affiche la liste que si `showDetails` vaut `true`.
- `<> ... </>` (Fragment) permet de retourner plusieurs éléments sans ajouter de `<div>`.





Étape 6 : Événements et liens externes

On ajoute deux boutons à chaque carte :

- **Wikipédia** : un gestionnaire d'événement construit l'URL à partir du nom et l'ouvre dans un nouvel onglet ;
- **Google Maps** : un simple lien vers l'URL stockée dans la clé `google_map` des données.

src/components/Card.jsx (version finale)

```
import Details from "./Details";

function Card({ wonder }) {
  const informations = {
    location: wonder.location,
    type: wonder.type,
    size: wonder.size,
  };

  const handleWikipediaClick = () => {
    const wikipediaUrl =
`https://fr.wikipedia.org/wiki/${encodeURIComponent(wonder.name)}`;
    window.open(wikipediaUrl, "_blank");
  };

  return (
    <div className="col">
      <div className="card h-100">
        <img src={wonder.image} className="card-img-top" alt={wonder.name}
             height={200} style={{ objectFit: "cover" }} />
        <div className="card-body">
          <h5 className="card-title">{wonder.name}</h5>
          <p className="card-text">{wonder.desc}</p>
          <Details informations={informations} />
          <div className="d-flex gap-2 mt-3">
            <button className="btn btn-primary" onClick={handleWikipediaClick}>
              Wikipédia
            </button>
            <a href={wonder.google_map} target="_blank" rel="noreferrer"
               className="btn btn-success">
              Google Maps
            </a>
```

```
          </div>
        </div>
      </div>
    </div>
  );
}

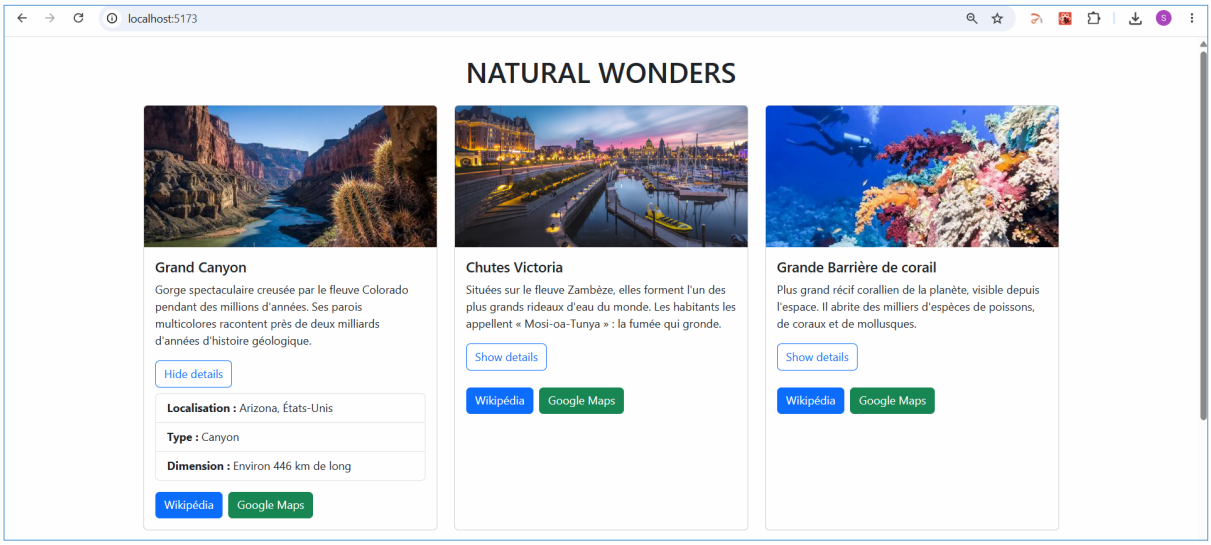
export default Card;
```

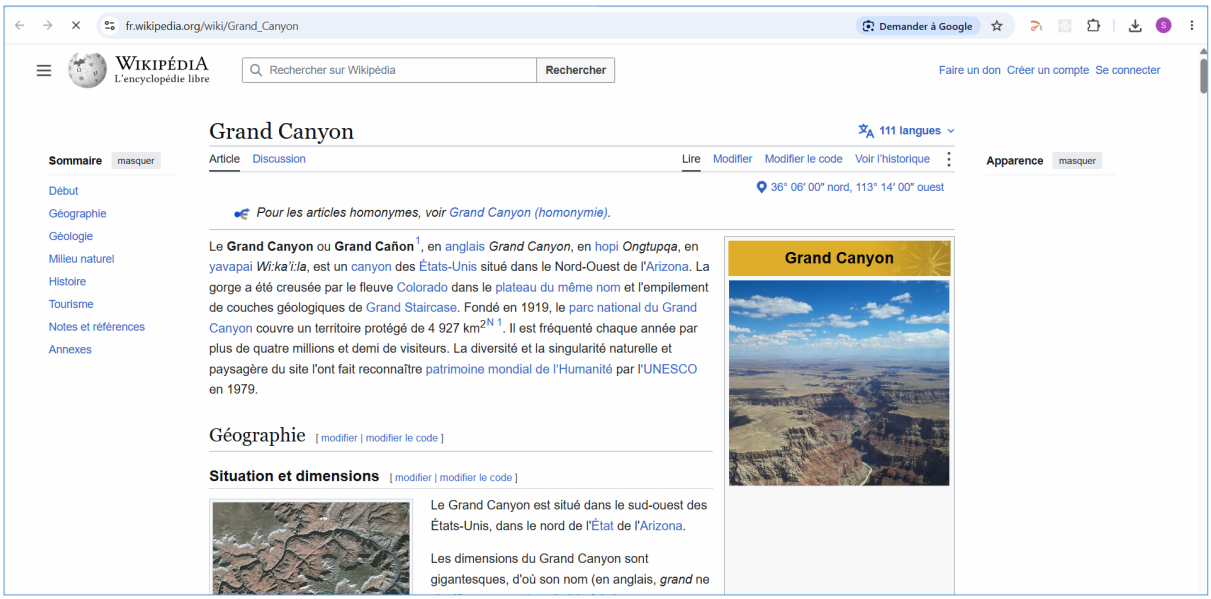
À retenir

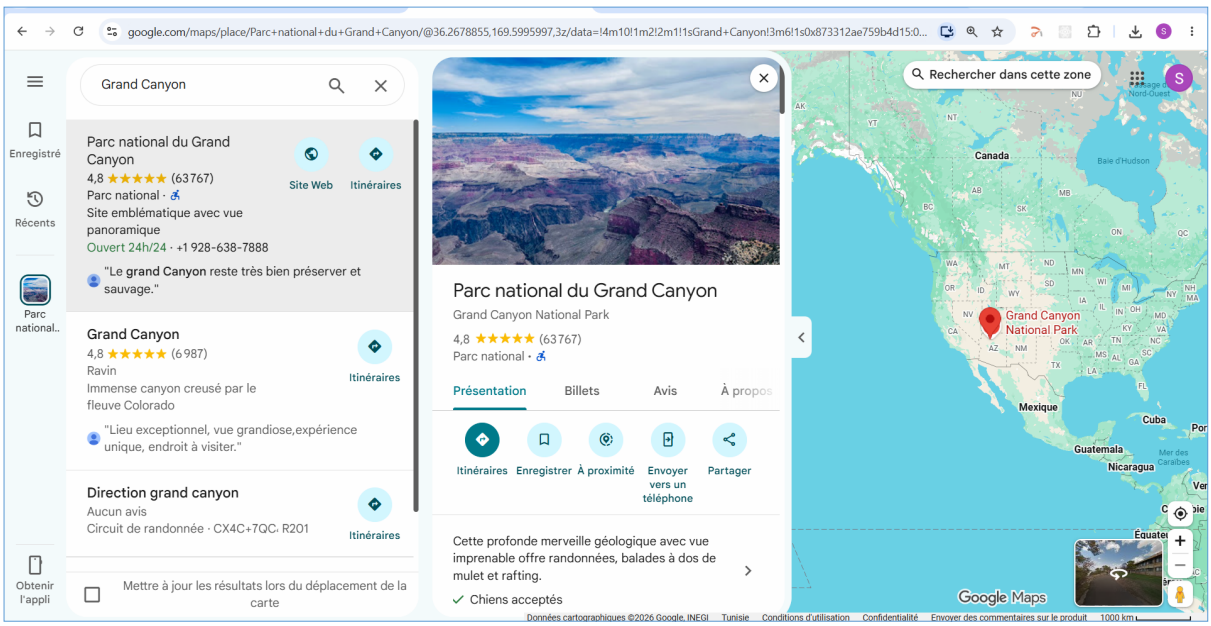
• Les accents graves (backticks, template literal) permettent d'insérer une variable dans une chaîne avec `${...}`.

• `encodeURIComponent()` remplace les espaces et caractères spéciaux : « Mont Everest » devient `Mont%20Everest`.

• On écrit `onClick={handleWikipediaClick}` (sans parenthèses) : on passe la fonction, on ne l'exécute pas tout de suite. S'il faut passer un argument, on écrit `onClick={() => maFonction(id)}`.







Étape 7 : Formulaire de connexion contrôlé

Dans un **formulaire contrôlé**, la valeur de chaque champ est stockée dans un état React. Le principe tient en 3 éléments :

Élément	Rôle
`useState("")`	Crée l'état du champ, vide au départ.
`value={username}`	Le champ affiche la valeur de l'état.
`onChange={(e) => setUsername(e.target.value)}`	À chaque frappe, l'état est mis à jour avec la valeur saisie (`e.target.value`).

Installer SweetAlert2, une librairie d'alertes plus élégantes que `alert()` (https://sweetalert2.github.io/) :

```
npm install sweetalert2
```

Créer le fichier `src/components/Login.jsx` :

src/components/Login.jsx

```
import { useState } from "react";
import Swal from "sweetalert2";

function Login() {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault(); // empêche le rechargement de la page

    if (username === "admin" && password === "1234") {
      Swal.fire({ icon: "success", text: "Connexion réussie !" });
    } else {
      Swal.fire({ icon: "error", title: "Erreur", text: "Identifiants incorrects" });
    }
  };

  return (
    <form onSubmit={handleSubmit} className="mt-5 mx-auto" style={{ maxWidth: "400px" }}>
      <h3>Connexion</h3>
      <div className="mb-3">
        <label htmlFor="username" className="form-label">Login</label>
        <input id="username" type="text" className="form-control"
               value={username} onChange={(e) => setUsername(e.target.value)} />
      </div>
      <div className="mb-3">
        <label htmlFor="password" className="form-label">Password</label>
        <input id="password" type="password" className="form-control"
               value={password} onChange={(e) => setPassword(e.target.value)} />
      </div>
      <button type="submit" className="btn btn-primary">Se connecter</button>
    </form>
  );
}

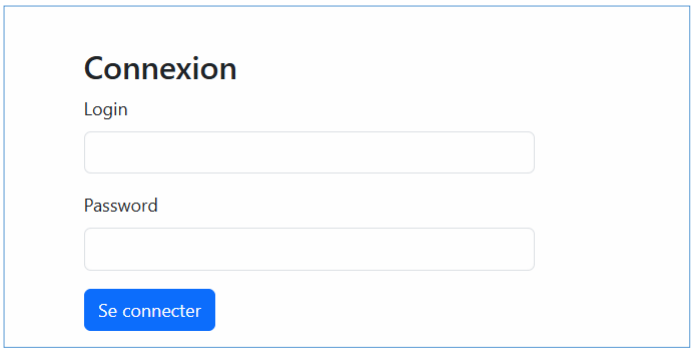
export default Login;
```

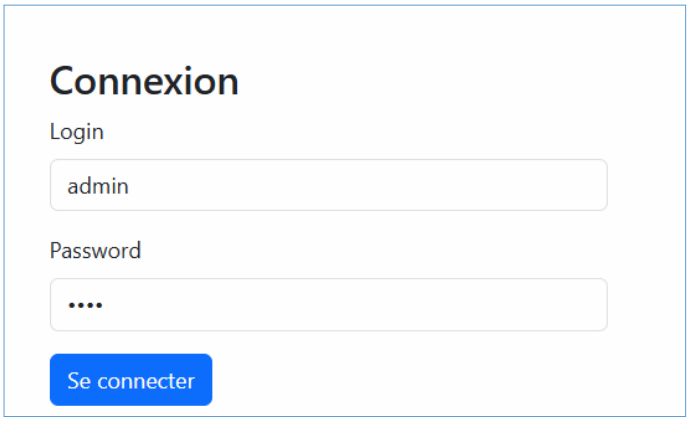
Dans `App.jsx`, importer et afficher le composant sous la grille :

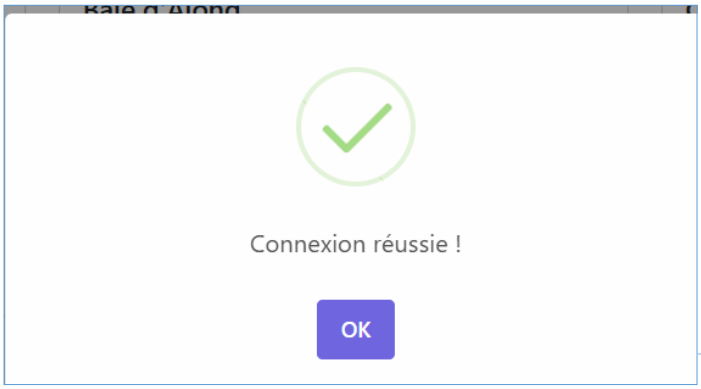
src/App.jsx (ajout)

```
import Login from "./components/Login";
...
      </div>  {/* fin de la grille */}
      <Login />
    </div>
```

> **Remarque**
> En JSX, l'attribut `for` d'un `<label>` s'écrit `htmlFor`, comme `class` s'écrit `className`.







Récapitulatif

Arborescence finale du projet :

```
natural-wonders/
├── public/
│   └── images/          (grand-canyon.jpg, victoria.jpg, ...)
└── src/
    ├── components/
    │   ├── Card.jsx     (carte + boutons Wikipédia / Google Maps)
    │   ├── Details.jsx  (détails + useState)
    │   └── Login.jsx    (formulaire contrôlé + SweetAlert2)
    ├── Data.jsx         (tableau WondersArr)
    ├── App.jsx          (titre + grille + Login)
    └── main.jsx         (import de Bootstrap)
```

Notions vues :

Notion	Où ?	Exemple
Composant	Card, Details, Login	`function Card() { return (...) }`
Props	App → Card → Details	`<Card wonder={wonder} />`
Déstructuration	Card, Details	`const { location, type } = informations`
Liste + key	App	`WondersArr.map(w => <Card key={w.id} ... />)`
useState	Details, Login	`const [x, setX] = useState(false)`
Rendu conditionnel	Details	`{showDetails && <ul>...</ul>}`
Événement	Card, Details	`onClick={handleWikipediaClick}`
Formulaire contrôlé	Login	`value={...} onChange={...}`

Travail à faire

Exercice 1 : Ajouter une merveille

Ajouter une 7e merveille naturelle dans `Data.jsx` (par exemple le Sahara, les aurores boréales ou l'Amazonie) avec toutes les clés. Vérifier qu'elle s'affiche sans modifier aucun composant.

Corrigé

Dans `src/Data.jsx`, ajoutez un 7e objet à la fin du tableau `WondersArr` (après l'objet `id: 6`, sans oublier la virgule) :

```
  {
    id: 7,
    name: "Sahara",
    desc: "Plus grand désert chaud du monde, il s'étend sur une dizaine de pays d'Afrique du Nord. Ses paysages alternent dunes de sable, plateaux rocheux et oasis.",
    image: "/images/sahara.jpg",
    location: "Afrique du Nord",
    type: "Désert",
    size: "Environ 9 millions de km²",
    google_map: "https://www.google.com/maps/search/?api=1&query=Sahara",
  },
```

Placez aussi l'image `sahara.jpg` dans `public/images`. La carte s'affiche sans toucher aux composants, car `App.jsx` parcourt tout le tableau avec `map()`.

Exercice 2 : Bouton « J'aime »

Dans `Card.jsx`, ajouter un état `likes` initialisé à 0 et un bouton « J'aime (n) » qui incrémente le compteur à chaque clic.

Corrigé

Dans `src/components/Card.jsx` :

```jsx
import { useState } from "react";
import Details from "./Details";

function Card({ wonder }) {
  const [likes, setLikes] = useState(0);

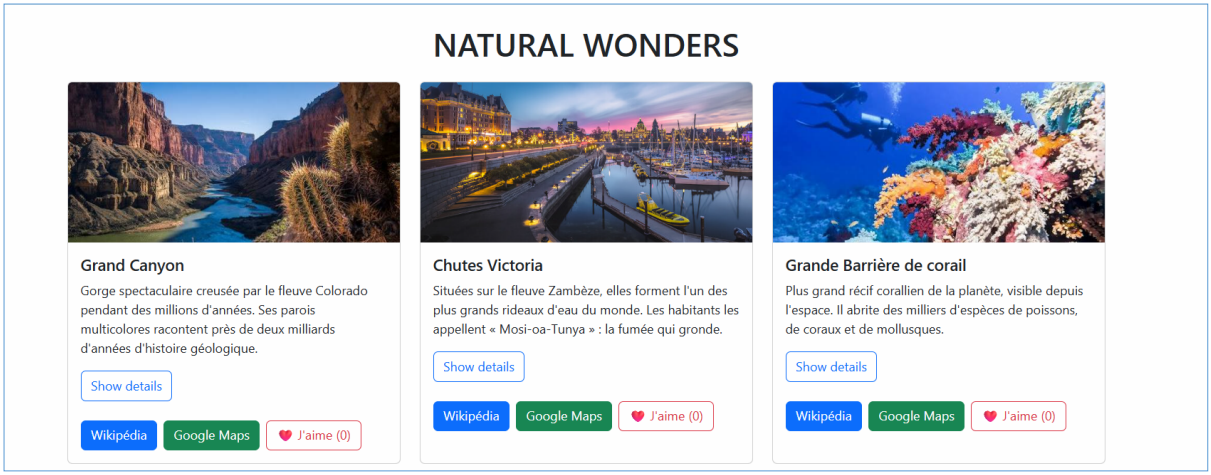
  const informations = {
    location: wonder.location,
    type: wonder.type,
    size: wonder.size,
  };

  const handleWikipediaClick = () => {
    const wikipediaUrl = `https://fr.wikipedia.org/wiki/${encodeURIComponent(wonder.name)}`;
    window.open(wikipediaUrl, "_blank");
  };

  return (
    <div className="col">
      <div className="card h-100">
        <img src={wonder.image} className="card-img-top" alt={wonder.name}
            height={200} style={{ objectFit: "cover" }} />
        <div className="card-body">
          <h5 className="card-title">{wonder.name}</h5>
          <p className="card-text">{wonder.desc}</p>
          <Details informations={informations} />
          <div className="d-flex gap-2 mt-3 flex-wrap">
            <button className="btn btn-primary" onClick={handleWikipediaClick}>
              Wikipédia
            </button>
            <a href={wonder.google_map} target="_blank" rel="noreferrer"
               className="btn btn-success">
              Google Maps
            </a>
            <button className="btn btn-outline-danger" onClick={() => setLikes(likes + 1)}>
              ❤ J'aime ({likes})
            </button>
          </div>
        </div>
      </div>
    </div>
  );
}

export default Card;
```

Chaque carte a son propre état `likes` : un clic sur une carte ne change pas le compteur des autres.



Exercice 3 : Barre de recherche

Dans `App.jsx`, ajouter un champ texte contrôlé (état `search`). N'afficher que les merveilles dont le nom contient le texte saisi.

> **Indice**
>
> Utiliser `filter()` avant `map()` : `WondersArr.filter(w => w.name.toLowerCase().includes(search.toLowerCase())).map(...)`

Corrigé

Dans `src/App.jsx` :

```
import { useState } from "react";
import Card from "./components/Card";
import Login from "./components/Login";
import { WondersArr } from "./Data";

function App() {
  const [search, setSearch] = useState("");

  const filteredWonders = WondersArr.filter((w) =>
    w.name.toLowerCase().includes(search.toLowerCase())
  );

  return (
    <div className="container my-4">
      <h1 className="text-center mb-4">NATURAL WONDERS</h1>

      <input
        type="text"
        className="form-control mb-4"
        placeholder="Rechercher une merveille..."
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />

      <div className="row row-cols-1 row-cols-md-3 g-4">
        {filteredWonders.map((wonder) => (
          <Card key={wonder.id} wonder={wonder} />
```

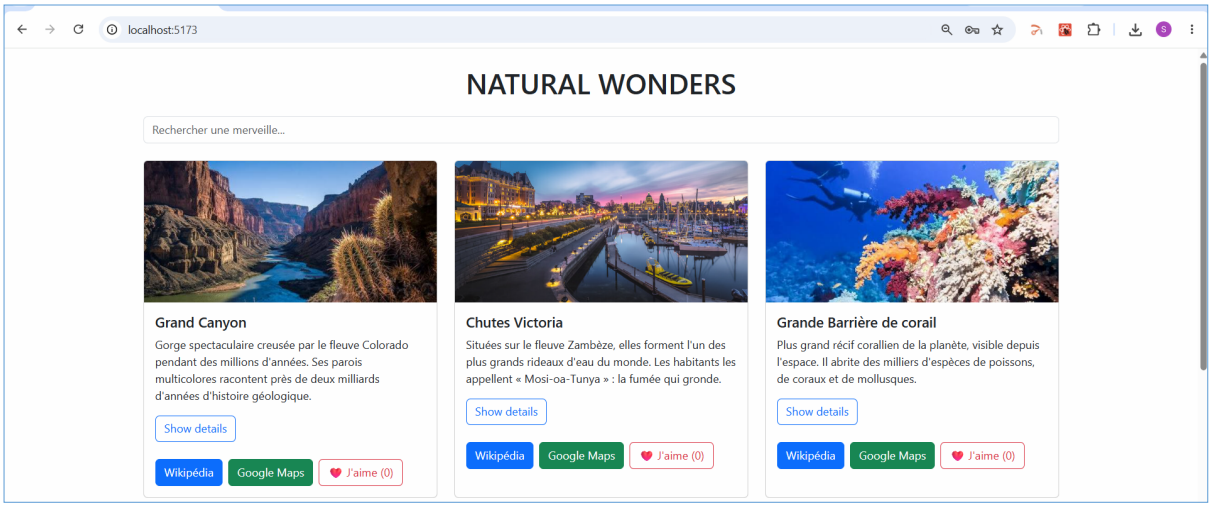
```
      ))}
    </div>

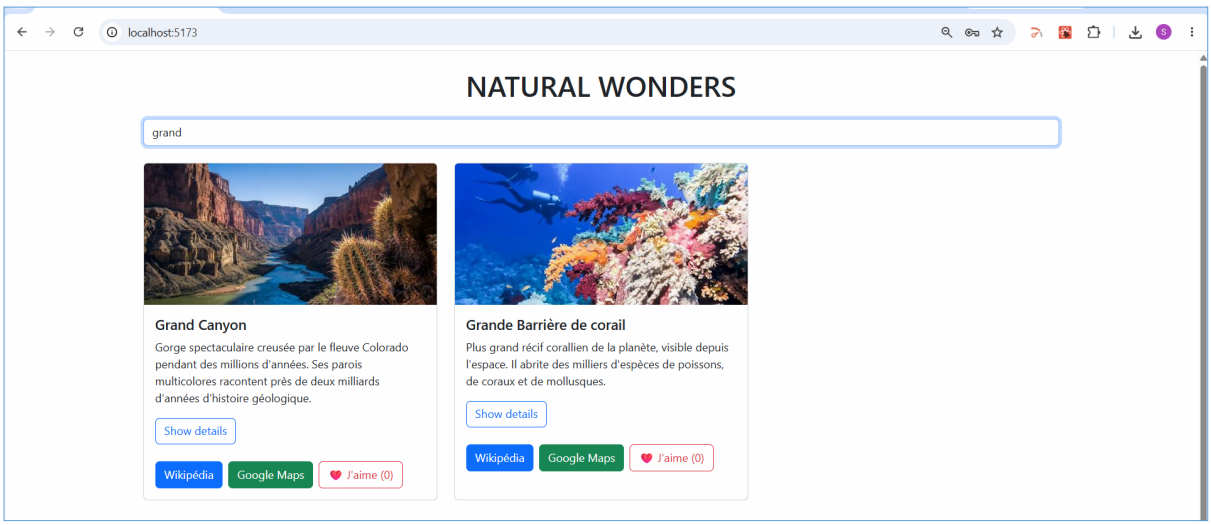
    {filteredWonders.length === 0 && (
      <p className="text-center text-muted mt-3">Aucune merveille trouvée.</p>
    )}

    <Login />
  </div>
 );
}

export default App;
```

`toLowerCase()` rend la recherche insensible aux majuscules : « chutes » trouve aussi « Chutes Victoria ». Le message « Aucune merveille trouvée » est un petit plus facultatif.





Exercice 4 : Message de bienvenue

Après une connexion réussie, remplacer le formulaire par le message « Bienvenue, admin ! ». Indice : ajouter un état `isLogged` et utiliser le rendu conditionnel.

Corrigé

Dans `src/components/Login.jsx` :

```jsx
import { useState } from "react";
import Swal from "sweetalert2";

function Login() {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [isLogged, setIsLogged] = useState(false);

  const handleSubmit = (e) => {
    e.preventDefault();

    if (username === "admin" && password === "1234") {
      Swal.fire({ icon: "success", text: "Connexion réussie !" });
      setIsLogged(true);
    } else {
      Swal.fire({ icon: "error", title: "Erreur", text: "Identifiants incorrects" });
    }
  };

  if (isLogged) {
    return (
      <div className="alert alert-success text-center mt-5">
        <h3>Bienvenue, {username} !</h3>
      </div>
    );
  }

  return (
    <form onSubmit={handleSubmit} className="mt-5 mx-auto" style={{ maxWidth: "400px" }}>
      <h3>Connexion</h3>
      <div className="mb-3">
        <label htmlFor="username" className="form-label">Login</label>
        <input id="username" type="text" className="form-control"
               value={username} onChange={(e) => setUsername(e.target.value)} />
      </div>
      <div className="mb-3">
        <label htmlFor="password" className="form-label">Password</label>
        <input id="password" type="password" className="form-control"
               value={password} onChange={(e) => setPassword(e.target.value)} />
      </div>
      <button type="submit" className="btn btn-primary">Se connecter</button>
    </form>
  );
}

export default Login;
```

Quand `isLogged` vaut `true`, le composant affiche le message à la place du formulaire. On aurait aussi pu écrire `{isLogged ? <message> : <form>}` dans un seul `return`.

